Poisoning the Deception Loop: Label Conflict in Honeypot-Fed Intrusion Detection

Harshit Chhabi
Vellore Institute of Technology
Vellore, India
harshitchhabi@gmail.com

ABSTRACT

Honeypot-fed intrusion detectors retrain on decoy traffic auto-labeled malicious by policy, on the argument that anything a decoy sees is unsolicited and therefore free ground truth. That argument assumes the label is trustworthy because the source is a decoy; it is not, whenever the attacker knows or can reliably infer the labelling policy, since it can then choose exactly what the decoy sees. We characterise the resulting attack on a modern, flow-based retraining loop: one mechanism, a label that contradicts its neighbourhood, that a fixed detection threshold surfaces as false positives and a recalibrated one surfaces as lost detection, not two independent effects. The false-positive form needs near-duplicate mimicry fidelity and at least 10–20% of the training set poisoned, in both models we evaluate, after Holm-Bonferroni correction across the full grid we test. The detection-loss form needs that same fidelity in both models, but for one of them, random forest, we further confirm with a dedicated, higher-powered follow-up that it also holds across a wide range of lower mimicry fidelity (realized distance 0.25–1.05), not only at copy-level fidelity — an effect a first, larger and lower-powered test could not resolve and that we report as confirmed once a properly targeted test could. A generic label-consistency defense fails structurally, because the poison agrees with the defender's own labelling policy; the cost-of-influence defense we set out to propose does not beat a matched no-skill baseline, a negative result we report rather than paper over. A one-line cap on the honeypot's effective training-set share is Pareto-undominated over the recovery-retention configurations we evaluate and costs almost nothing to run. We additionally measure that CICIDS2017's benign traffic is narrow enough that an attacker reaches damaging fidelity at low marginal traffic-generation cost, which bounds what this widely used benchmark can demonstrate about the attack's cost in general.

1 INTRODUCTION

A machine-learning intrusion detector needs labeled malicious traffic to train on, and labeled malicious traffic is expensive: someone has to attack the network, or wait for someone else to, and then confirm what happened. Honeypots offer an appealing shortcut. A decoy has no production purpose, so nothing legitimate should ever reach it; whatever does is unsolicited by definition, and can be auto-labeled malicious with no human in the loop. Feed that traffic into a detector's training set and retrain periodically, and the detector adapts to new attack traffic without anyone hand-labeling a single flow. This design is not hypothetical. Recent systems build exactly this loop: Matcheswala and Javed [13] pairs an adaptive honeypot with a network traffic classifier trained on what the honeypot collects, and Lanka et al. [11] applies machine-learning analysis to honeypot-collected data for threat detection. Neither system asks what happens if the traffic reaching the decoy is not what the auto-labelling policy assumes it is.

The assumption the whole design rests on is that the label is trustworthy *because* the source is a decoy. That assumption does not survive contact with an adversary who knows, or can reliably infer, the policy. The labelling rule — "everything the honeypot sees is malicious" — is not a secret that technical means protect: a honeypot has to be reachable to do its job, and the design itself is describable in a paper, a product page, or open-source code, so an attacker does not need privileged access to learn it, only the kind of knowledge that is often freely available (we make this an explicit axis in Section 3 rather than an unstated assumption). An attacker who has that knowledge does not need to compromise anything to write into the defender's training set: sending traffic to the decoy *is* writing to the training set, with a label the defender chooses to trust unconditionally. The honeypot is not a passive sensor in this design; it is a write channel the defender opens on purpose, and the poison budget available through it, once reached, is not a fraction an attacker has to fight for — it is the entire honeypot-derived share of the training set that the attacker's traffic is admitted into (Section 3).

This paper characterises that channel on a modern, flow-based network intrusion detection loop: seed data plus honeypot traffic, retrained periodically, scored against held-out production traffic. We confirm first that the loop does what it claims when the honeypot sees genuine attacks — the honest case, S0, has to work before an attack on it means anything (Section 6). We then attack it with A1, benign mimicry: point the same traffic generator that produces legitimate benign flows at the honeypot instead of production, and let the auto-labeller do the rest. The result is one mechanism — a label that contradicts the neighbourhood it sits in — that surfaces as false positives under a fixed detection threshold or as lost detection under a recalibrated one, not two independent effects; recalibrating relocates where the damage shows up, it does not remove it. The two surfaces need different fidelity: on CICIDS2017, the false-positive surface needs near-duplicate mimicry fidelity and collapses to nothing beyond it, while the detection-loss surface, confirmed for one of our two models, holds across a much wider range of mimicry fidelity (realized distance 0.25–1.05, not just near-duplicate). Neither needs model access or knowledge of the poison ratio, and neither costs the attacker more than generating traffic that already resembles what the network produces — a low marginal cost under the conditions we test, not a formally zero one. We hold every quantitative claim in this paper to a pre-registered,

multiple-comparison-corrected significance bar, and where a pattern in the data does not clear that bar we report it as unconfirmed rather than round it up to a finding.

We then evaluate defenses. A generic label-consistency check — does this row's stamped label agree with a model fit on the loop's own data — fails structurally against this attack, because the poison is not inconsistent with anything the loop itself believes; it agrees with the defender's own labelling policy, which is exactly what makes it hard to catch from inside the loop. Cost-of-influence weighting, which we set out to propose as this paper's defense, does not survive its own pre-registered evaluation: it fails a sensitivity criterion fixed in advance and does not beat a matched no-skill baseline. What does work, and costs almost nothing to deploy, is capping the honeypot's effective share of the training set — a one-line change that needs no label information, no trusted reference data beyond the cap value itself, and no estimate of the poison ratio, and that no alternative we test dominates. Along the way, measuring the attack on CICIDS2017 exposes a property of the dataset itself: its benign traffic is narrow and repetitive enough that an attacker gets most of the way to the damaging fidelity level at low marginal traffic-generation cost, simply by replaying the pool's own statistics with no perturbation at all — a fact about this benchmark's degeneracy, not a general claim about production network traffic.

Contributions

(1) A threat model for auto-labelled honeypot data as an attacker-controlled write channel into IDS training, characterised — mechanism, fidelity requirement, matched null controls — on a modern honeypot-fed retraining loop: the design that older signature-generator attacks targeted in a different form [15, 18], and that current honeypot-plus-ML systems reintroduce without evaluating the threat (Section 3).
(2) The attack characterised as one mechanism, surfaced as either false positives or lost detection depending on the threshold policy in force, not two independent channels; damage reported as a function of measured mimicry fidelity (twin fraction, not a jitter knob or a median distance too coarse to resolve the effect), held to a pre-registered, multiple-comparison-corrected significance bar throughout, including a dedicated, higher-powered follow-up that confirms one model's detection-loss damage extends well past copy-level fidelity, where a larger but lower-powered test could not resolve it (Section 6).
(3) The policy-consistency blind spot: why a defense that checks a label's consistency against the loop's own data cannot catch poison that agrees with the loop's own labelling policy, and why a defense anchored to trusted data outside the loop can (Section 7).
(4) A defense comparison against a proper no-skill baseline: cost-of-influence weighting, our original proposal, is a measured negative result that does not beat it; a one-line cap on the honeypot's training-set share is undominated by every alternative tested and costs almost nothing to run (Section 7).
(5) A measurement of CICIDS2017's near-degeneracy under standard flow features, including that the attack's damage on this dataset traces to near-duplicate, not exact-duplicate, benign traffic, and what that implies for which claims this benchmark can and cannot support (Section 5).

2 BACKGROUND AND RELATED WORK

Honeypot-fed ML and closed-loop retraining. Systems that feed honeypot traffic into a machine-learning detector are an active design pattern, but almost all of them run the dependency the other way from what we attack: the honeypot supplies data that trains or informs a classifier, and the classifier's job is to detect attacks, not to question the honeypot's labels. Two papers from 2024 retrain a network-traffic classifier on honeypot-collected data as part of an active defense: Matcheswala and Javed [13] pairs an adaptive honeypot with a network traffic classifier trained on what it collects, and Lanka et al. [11] applies AI-driven analysis to honeypot data for threat detection. Neither evaluates whether an attacker who knows the labelling policy can shape what the classifier learns — the closed loop is assumed trustworthy because its input is a decoy, which is exactly the assumption this paper tests. Yugai and Moskvin [25] and Abdou et al. [1] put "honeypot" and "machine learning" together most literally but point the other way: the former uses a honeypot plus a classifier to detect adversarial evasion attempts against an IDS, and HoneyModels deploys ML models *as* decoys to attract and study adversarial examples against the model itself. In neither case is the honeypot's output treated as an adversarial input channel into a defender's training set.

Adaptive and LLM-driven honeypots. A separate, larger literature makes honeypots themselves smarter — more convincing, harder to fingerprint, cheaper to operate — without changing what happens to their output. LLM-backed shells and terminals raise interaction fidelity [8, 16, 24], reinforcement learning tunes engagement policy [9], and a recent survey maps the space [5]. Srinivasa et al. [22] studies an attacker fingerprinting honeypots to decide whether to engage them at all — relevant to selective disclosure (A4) and persona defenses (D3), which this paper scopes out, but orthogonal to A1: fingerprinting decides whether traffic reaches the decoy, not what happens to the label once it does. Vargas Yáñez and Tobón [23] is recent (2026) evidence that honeypot output is not automatically trustworthy even on its own terms: prompt injection against an LLM-based honeypot classifier pushes its evasion rate to 54.8% absent a dedicated defense. The mechanism is different from ours — that attack corrupts what a honeypot's own classifier concludes about an interaction (evasion), where A1 leaves the honeypot's record untouched and exploits the policy applied to it afterward (poisoning) — but it is the same underlying point: a honeypot's output is a channel an attacker who understands it can shape, not a ground truth that arrives for free. None of this work asks whether a smarter honeypot produces safer training labels; convincing the honeypot's human or automated operator is a different problem from convincing the auto-labeller downstream.

Poisoning of continuously-retrained classifiers. The mechanism we attack — an automated learner retrained on traffic an adversary can shape — is not new; only "honeypot" and "LLM" are. Newsome et al. [15] and Perdisci et al. [18] attack Polygraph's

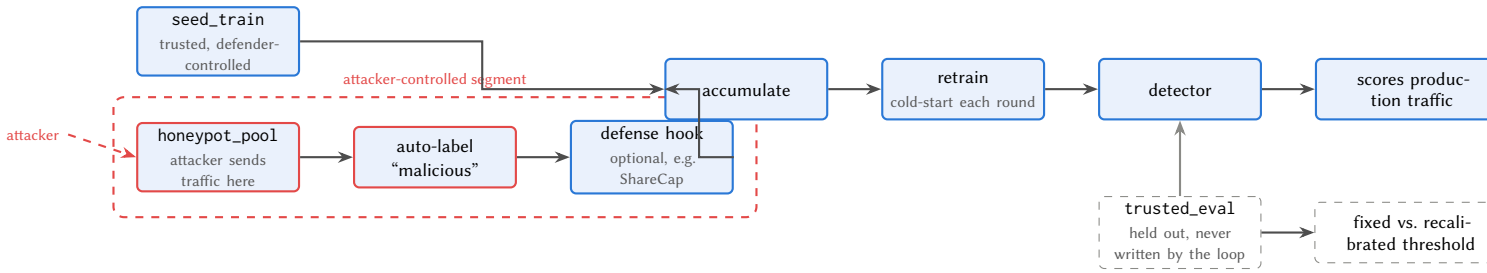


Figure 1: The retraining loop. Everything the honeypot logs is auto-labeled malicious by a policy an attacker who knows the design can predict, and feeds the same `accumulate` step as trusted seed data (dashed red box: the attacker-controlled segment — the only traffic the attacker chooses is what the decoy sees, and that traffic becomes training data unconditionally). `trusted_eval` is held out and never written by the loop; every result in this paper is scored against it under both a fixed and a recalibrated detection threshold.

worm-signature learner, trained on flows a honeynet or IDS flags as suspicious, with crafted noise; the closest older analog to this paper, but aimed at evading detection of one worm through a signature matcher, not at raising false positives on unrelated production traffic through a retrained statistical classifier, and without a persistent multi-round loop that accumulates a poison share over time. Nelson et al. [14] degrades a continuously-retrained Bayesian spam filter through a write channel — user feedback that becomes training data — that keeps retraining on the stream it poisons; the closest continuous-retraining analog, but the channel is inbox placement, not a honeypot with a deterministic labelling policy an attacker who knows the design can predict in advance. Rubinstein et al. [20] and Kloft and Laskov [10] poison online/continuously-updated anomaly detectors with traffic injected directly into what the detector monitors; structurally our loop, but with no honeypot, no deception element, and no knowable labelling policy — the attacker there must evade the detector's own scoring to get poison admitted, rather than simply reaching a decoy that labels everything malicious by design once the attacker knows that is the design. Biggio et al. [3] formalizes poisoning as bilevel optimization against a retrained SVM, a generic method this paper's threat model does not need: A1 requires no model access, only knowledge of the honeypot's labelling semantics (Section 3).

Poisoning defenses. Barreno et al. [2] lays out the taxonomy this whole line of work sits inside — causative and exploratory attacks, targeted and indiscriminate, and the RONI defense — as a general framework with no honeypot-specific threat model; this paper specializes one causative instance end to end. Paudice et al. [17] sanitizes labels by *k*NN vote against a trusted reference, the same mechanism as this paper's kNN comparator baseline (Section 7), but evaluated against generic label-flipping poisoning rather than a honeypot-sourced channel whose poison is consistent with the defender's own labelling policy; it does not surface the policy-consistency blind spot this paper reports for loss-based filtering, because its threat model has no "the label is wrong but policy-consistent" case to fail on. Biggio and Roli [4] and Cinà et al. [6] survey the broader adversarial-ML landscape without a honeypot-specific instance.

IDS dataset critiques. CICIDS2017 [21] has been criticised before for labeling and extraction errors [7, 12, 19]; this paper adds a distinct, narrower finding — every family with usable evaluation volume, benign traffic included, sits far below a leakage threshold in flow-feature space regardless of how the data is partitioned (Section 5) — that constrains what any learning claim on this dataset, ours included, can support.

The gap. We found no prior work evaluating this specific write-channel threat model: a honeypot's auto-generated label treated as an untrusted input to a system that keeps learning from it, rather than as ground truth. We do not claim this search was exhaustive, only that the closest work we found either attacks a different target (a signature matcher, a spam filter, an anomaly detector scored on real production traffic it must evade) or builds a better honeypot without asking what happens to its output downstream. This paper characterises the attack this gap implies on a modern, flow-based NIDS retraining loop: the mechanism, the fidelity each surface of the attack needs, a defense comparison against a proper no-skill baseline, and a dataset-specific measurement of why the attack reaches damaging fidelity at unusually low cost on the benchmark most available to reproduce it on.

3 THREAT MODEL

The systems we attack retrain on two sources: a trusted seed set the defender controls, and whatever traffic reaches a honeypot, auto-labelled malicious by policy — anything a decoy sees is unsolicited by definition, so the argument goes, and therefore free ground-truth-malicious data. That argument holds only if the label is trustworthy because the source is a decoy. It is not, whenever the attacker knows or can reliably infer that policy: it can then choose exactly what the decoy sees, with a label it knows the defender will trust. The honeypot becomes a write channel into the defender's training set, opened by the defender's own design. The poison budget is not a fraction the attacker has to fight for against a defended channel; it is the entire honeypot-derived share of the training set that the attacker's admitted traffic reaches, by construction of the design — what fraction of *all* traffic entering retraining that share amounts to is a separate, measured quantity (realized poison ratio, below).

Attacker capability. The attacker can send arbitrary traffic to the honeypot, but cannot touch `seed_train`, `trusted_eval`, or the model directly — every effect it has on the deployed detector

routes through what the honeypot logs. The pool it draws from is two-class, sized so poison ratios up to 50% of the training set are reachable with fresh rows rather than resampled duplicates; for a tree model, a resampled duplicate is closer to upweighting a sample already present than to injecting new poison, which would understate what the attack can do.

Attacker knowledge. We separate what the attacker needs to know along four axes, rather than bundling them into one assumption. *Labelling-policy knowledge* is the one the attack depends on most, and we model it at four levels: *unknown* (the attacker has no information about how honeypot traffic is used); *inferred* (the attacker suspects or deduces, from observing honeypot behaviour or general knowledge of how such systems are built, that anything the decoy logs is treated as malicious ground truth); *known* (the attacker knows this outright, because the design is published — an academic description, vendor documentation, an open-source implementation — or because a honeypot that must be reachable to do its job cannot keep that much secret); and *known + retraining-aware* (the attacker additionally knows or tracks the defender's retraining schedule well enough to time its traffic). A1 as implemented assumes the *known* level: it requires that the attacker knows, not merely suspects, that honeypot traffic is auto-labelled malicious and fed into training, but it needs nothing beyond that — no retraining-schedule awareness, no behavioural inference from a black-box honeypot. This is a real assumption, not a vacuous one: an attacker who only observes the honeypot's external behaviour, without access to a description of the design, would have to infer the policy rather than know it, and we do not evaluate how much harder that makes the attack. *Model knowledge*: none. A1 never queries the detector, computes a gradient against it, or otherwise interacts with it; everything it needs routes through the honeypot. *Benign-distribution knowledge*: A1 as implemented draws its poison from benign *feature rows* already in the attacker's reach in our experiments — the same pool the defender's own traffic generator uses — which assumes access to the benign class's statistics in the detector's own feature space. Realizing equivalent mimicry from packet-level traffic alone, without that feature-space access, is not demonstrated here and is left as future work (Section 8). *Retraining-schedule knowledge*: none needed. The attack is agnostic to when retraining happens; it only needs its traffic admitted into whatever batch is next retrained on.

Attacker goal. Raise the detector's false-positive rate on production traffic, or suppress its detection of a real attack. **A1 (benign mimicry)**, this paper's subject, does the first: point the same kind of traffic generator that produces legitimate benign flows at the honeypot instead of production, and let the auto-labeller stamp the result malicious. **S0 (clean)** — the honeypot sees genuine attacks, correctly labelled — is not an attack; it is the sanity check this design has to pass before any attack against it means anything (Section 6). **A2 (boundary drift)** and **A4 (fingerprint-and-split)** are scoped out: A2 needs a gradual multi-round adversary this paper does not build, and A4's controlled measurement arm cannot occur naturally on CICIDS2017, where every family runs on a single day (Section 4).

Realized poison ratio. Where this paper reports a poison ratio, it means the operational quantity: attacker-admitted honeypot samples divided by all samples entering that round's retraining (seed plus honeypot, admitted after any defense hook). It is realized, not nominal — the target ratios named in Section 4 are generation targets that the loop's batch sizing is tuned to reach by the final round, not a guarantee independent of what else is happening in the honeypot. In particular this ratio depends on two things we do not model: how much background (non-adversarial) traffic the honeypot also receives, which would dilute the attacker's share of what enters training, and any rate limiting the honeypot or its ingestion pipeline applies, which would cap how much poison the attacker can inject per round regardless of how much traffic it sends. Every ratio reported in this paper is the realized value measured directly from the loop's own admitted batches, not back-computed from a generation target, but it is realized against a honeypot with no competing traffic and no rate limit — a best case for the attacker that a deployed honeypot need not offer.

Capability axis. A1's only free parameter is how closely the attacker's traffic resembles genuine production benign traffic. We do not assume this from a generation setting; we measure it directly as the realized nearest-neighbour distance from the injected poison to trusted benign traffic (Section 4), and report damage as a function of that measured distance rather than of the knob that produced it. The median of this distance turns out to be too coarse a summary: it changes by about 1% across the exact step where the FPR channel's damage collapses from catastrophic to noise-floor (Section 6). We additionally report *twin fraction* — the share of a poison batch whose nearest trusted- benign neighbour sits within a fixed RMS delta (0.0015, chosen from where the distribution's lower tail moves between the jitter steps that separate damaging from non-damaging poison, Section 6) — which does track the collapse and is the primary axis for the FPR figure; median distance is kept as a secondary statistic since it still separates points once twin fraction has gone uniformly to zero.

4 METHODOLOGY

4.1 The loop

Each round: the adversary generates a batch of flows and a per-batch cost record (flows, packets, bytes, flow-time); the auto-labeler stamps every row in the batch (Section 3); a defense hook, when one is installed, sets a per-row `sample_weight` and may quarantine the whole batch; the batch is added to the accumulated training set; the model is **retrained from scratch** on seed data plus every admitted batch to date; the fresh model is scored on `trusted_eval`. Retraining cold-start each round, rather than warm-starting from the previous round's model, keeps rounds comparable and prevents poisoning dynamics from being confounded with optimization path dependence. Two knobs govern how honeypot data accumulates. *Retention* is `accumulate` by default: every admitted batch is kept for the rest of the run, which is both the design the honeypot-retraining literature proposes ("continuous adaptation") and the worst case for the defender, since influence only grows. *Budget mode* is `fixed_ratio`: the per-round batch size is set so the honeypot's share of the training set reaches a target ratio by the final round, so every point on a damage curve has a known, comparable poison share. No model in this codebase uses automatic class rebalancing (`class_weight="balanced"`, XGBoost's `scale_pos_weight`, oversampling, SMOTE). A1 works by injecting malicious-labeled

Table 1: Partition sizes (rows), near-duplicate grid 0.005. Attack rows have no `prod_benign` share.

Partition	CICIDS2017		Synthetic	
	benign	attack	benign	attack
seed_train	344,823	78,877	20,383	8,241
prod_benign	229,139	–	13,098	–
honeypot_pool	344,221	130,166	19,951	7,993
trusted_eval	229,695	52,054	13,262	4,535

volume, which shifts the class balance; an automatic rebalancer would respond to that shift and the measured attack effect would be entangled with a rebalancing artifact. `sample_weight` is the only channel by which any row's influence on the fit is adjusted, and every defense in Section 7 rides on it. Every fit seeds Python, NumPy and PyTorch and forces deterministic kernels. The model seed varies by round but is shared across every arm at a given (seed, round), so a paired, same-seed delta between an arm and the control isolates the treatment rather than retraining noise.

4.2 Datasets and partitions

Data comes from CICIDS2017 (`MachineLearningCVE`) and a synthetic generator that ships with the codebase, both partitioned identically into four disjoint sets: `seed_train` (the defender's trusted starting data), `prod_benign` (unused here), `honeypot_pool` (what the adversary can draw from and what genuine decoy traffic would supply), and `trusted_eval` (held out, never written to by the loop). The split is within each (day, family) block, not each whole day: benign rows go 30/20/30/20 to `seed_train`, `prod_benign`, `honeypot_pool` and `trusted_eval` respectively; attack rows go 40/40/20 to `seed_train`, `honeypot_pool` and `trusted_eval`. This gives every attack family a presence in every partition (`seen_both`), which a whole-day split cannot, since CICIDS2017 runs each family on its own day.

The CICIDS split is not temporal. The `MachineLearningCVE` CSVs carry no timestamp column; a timestamp is synthesized at one row per second in file order. Every cut, buffer and burst boundary computed on CICIDS in this paper is therefore a file-order cut, not a wall-clock one. Section 5 shows this distinction does not matter to the result: a uniformly shuffled re-split reproduces the same feature-space distances. Four eval-family arms are tracked, never folded into an aggregate: `seen_both` (in both seed_train and honeypot_pool, the literature's regime), `seed_only` (A4's controlled arm), `honeypot_only` (the loop's most interesting arm: a family the detector saw only via the decoy), and `novel` (in neither).

The synthetic generator emits data in the same shape as the real CSVs, with well-separated class geometry so a clean loop can demonstrably improve, and injects the same corruption classes (non-finite rates, negative durations, duplicate rows) so the cleaning pipeline is exercised the same way on both datasets. It has no model of attacker cost: a flow's duration, packet count and byte count are drawn from the same class-conditional distribution regardless of which scenario produced it.

4.3 Leakage guards

Three guards run on every partition build. (a) Near-duplicate rows are removed globally, before splitting, at grid resolution 0.005 (grid sweep and rationale in Appendix A). (b) Rows within a buffer of an internal cut are dropped (burst-level splitting, a stronger version of this guard, is also in the appendix; it does not change Section 5's result). (c) For every `trusted_eval` row, the nearest-neighbour distance in normalized feature space to its nearest `honeypot_pool` row of the same class is computed and reported for both classes — this is the metric behind every "realized distance" and "twin fraction" number in this paper. Why the resulting `leak_warning` is gated on the attack class alone, and what that does and does not imply about the benign side, is in Appendix A.

4.4 Attack generation and the distance metric, formally

Feature domain. Every canonical feature (23 total: duration, packet and byte counts, packet-length and inter-arrival statistics, flag counts, rate and ratio features) is a non-negative scalar — a count, a duration, a length, or a ratio of these. There are no categorical or protocol-enum fields in the feature vector this paper's models and adversaries operate on; protocol information (e.g. TCP vs. UDP) is not part of the canonical schema, so no categorical encoding or distance treatment is needed for it. Infinite and NaN cells (CICIDS2017 is documented to produce both, from byte/packet rates divided by a zero-duration flow) are replaced with NaN and the affected rows dropped during cleaning, upstream of everything below; rows with a negative value in a physically non-negative feature (a known CICIDS2017 corruption artifact) are dropped the same way. Every row the attack generator or the distance metric ever sees is already finite and non-negative.

Normalized feature space. A *normalizer* is fit once per dataset, on the training side only, never on validation or held-out rows. For feature k, let μ_k be the median and $\sigma_k = \max(\mathrm{IQR}_k, 0.1)$ the (floored) interquartile range of $\log(1 + x_k)$ over the fitting rows — the 0.1 floor exists because several CICIDS2017 flag-count features are near-constant in benign traffic, giving them a near-zero IQR that would otherwise blow up every row where the feature is non-zero. The transform is $z_k = \mathrm{clip}\left(\frac{\log(1+x_k)-\mu_k}{\sigma_k}, -25, 25\right)$, applied feature-by-feature; this is the space every realized-distance, twin-fraction, and guard-(c) number in this paper is computed in. The *RMS nearest-neighbour distance* between a row z and a reference set $\{z^{(i)}\}$ is $d(z) = \min_i \sqrt{\frac{1}{K}\sum_{k=1}^{K}(z_k - z_k^{(i)})^2}$ over the $K = 23$ canonical features — root-mean-square, not sum, so the metric does not grow with feature count and stays comparable across subsets.

Attack-generation function. A1's poison row x' is generated from a source row x (drawn from the honeypot pool's benign-labelled rows) in two steps. First, the jittered normalized representation: $z' = \mathrm{clip}\big(z(x) + \sigma_j\,\epsilon,\ -25,\ 25\big)$, where $z(x)$ is the normalizer's transform of x (Equation above, already clipped to ±25), $\sigma_j \geq 0$ is the *jitter* parameter (a generation knob, not a reported quantity — Section 3's capability axis is the *realized* distance this produces, measured after the fact, not σ_j itself), and $\epsilon \sim \mathcal{N}(0, I_K)$ is unit Gaussian noise, in normalized units, drawn fresh per row per

round. Second, inverting back to raw feature space: $x' = G(x, \sigma_j, \epsilon) = \text{clip}\big(\text{expm1}(\min(z' \odot \sigma + \mu,\ 25))\,,\ 0,\ 10^{10}\big)$, where $\odot$ is elementwise product with the normalizer's per-feature scale σ and μ its per-feature center; the inner $\min(\cdot, 25)$ caps the log1p-space argument before exponentiating (a separate clip from z''s own, needed because σ can be large enough that $z' \cdot \sigma$ would otherwise overflow float32 in expm1), and the outer clip to $[0, 10^{10}]$ floors every feature at its physical non-negative bound and caps an extreme rebound value. $\sigma_j = 0$ makes $z' = z(x)$ exactly, reproducing the source row — raw copy fidelity, the left end of every damage-versus-fidelity curve in this paper. An optional padding step, cost_padding = $m \geq 1$, multiplies the five attacker-cost features (duration, forward/backward packet and byte counts) by m *before* $z(x)$ is computed, simulating an attacker who makes its interaction look more expensive to buy weight under a cost-based defense (Section 7, Appendix A).

The 0.25 guard-(c) threshold is a chosen operating point, not a derived constant. It was picked to be comfortably above ordinary measurement noise in this normalized space and comfortably below where two rows would plausibly be considered "the same flow," not fit to any distribution. Its only role in this paper is binary: flagging whether a family's distances are small enough to call the partition degenerate for that family. The conclusion that holds under it — median distances of 0.0030–0.0177 across every family and every deduplication grid tested, all at least 14× below 0.25 (Table 3) — is not sensitive to where exactly the line is drawn: the same families are flagged at any threshold from roughly 0.02 up to well past 0.25 and down to an order of magnitude above the measured values, since the largest median distance measured anywhere in this paper's degeneracy table is 0.0177. Only a threshold tighter than about 0.02 — eight times stricter than the one used — would change which families the guard flags.

4.5 Threshold policy

No model uses a 0.5 cutoff. Each model's threshold is calibrated to a target FPR (0.01) on a validation split carved from its own training data. Two threshold modes are reported for every run, because A1's damage surfaces in different places under each: *fixed* calibrates once at round 0 and holds that threshold, so damage appears as rising FPR on trusted benign; *recalibrated* recalibrates every round on trusted benign, so FPR stays capped by construction and damage appears as collapsing TPR instead. On CICIDS2017 this calibration is not honest by default: the validation split is carved from training data that, by Section 5's measurement, is full of near-copies of what it is meant to hold out, so a threshold calibrated against it is optimistic and lands too low (control fixed-threshold FPR 0.046 against a 0.01 target). We calibrate instead only on validation rows with no same-class training twin within a distance $\tau = 0.1$ (the same per-feature RMS metric as guard (c)). This discards 94% of benign validation rows and moves the control's fixed-threshold FPR to 0.007. Every CICIDS number in this paper after Section 5 uses this calibration.

4.6 Arms

Every experiment compares an arm against a same-seed *control* (no honeypot ingestion, retrain on seed_train alone each round; the model seed still varies by round, so the control measures real retrain-to-retrain noise, not zero). **S0** (clean) feeds genuine attack rows, correctly labeled. **A1** (benign mimicry) feeds benign rows, jittered by a controlled amount, stamped malicious by the auto-labeler. Section 6 isolates A1's mechanism with three matched null controls, each removing exactly one candidate cause: `a1truth` (A1's own rows, correctly labeled — removes the label conflict), `s0j` (attack rows carrying A1's jitter, stamped malicious — same volume and class-prior shift, no label conflict), and `junk` (marginal-shuffled bulk, stamped malicious — volume with no structure at all).

4.7 Models, grids and metrics

Two models are trained per round: a random forest (25 trees, max depth 16) and XGBoost (60 trees), both at library defaults otherwise. Every result in this paper is 20 rounds, 5 seeds, unless stated otherwise. Each round retrains on a fixed subsample of the partitions in Table 1: up to 30,000 `seed_train` rows, evaluated against up to 40,000 trusted benign rows and up to 4,000 `trusted_eval` rows per attack family, drawn once per dataset (not per seed) so between-seed variance reflects model and adversary randomness rather than a moving evaluation target. The A1 poison-ratio grid is log-spaced (finer at the low end, where damage first appears if it appears at all): $\{0, 0.005, 0.01, 0.02, 0.05, 0.1, 0.2, 0.5\}$ generally; the damage-versus-distance sweep in Section 6 uses ratios $\{0.05, 0.1, 0.2, 0.5\}$ and jitter $\{0, 0.01, 0.03, 0.1, 0.3, 0.7, 1.5\}$, roughly log-spaced in *realized* distance. The jitter knob is a generation parameter, not what is reported: every A1 plot's x-axis is the *realized* mimicry distance — the median nearest-neighbour distance (guard-(c) metric, per-feature RMS, normalized space) from the injected poison rows to the full trusted benign set, measured directly, not assumed from the jitter setting. Section 7 reports two summary quantities against these arms. *Recovery*, $R = 1 - \overline{\Delta\text{FPR}}_{\text{defended}} / \overline{\Delta\text{FPR}}_{\text{undefended}}$, is measured on A1 at raw benign fidelity (jitter 0), poison ratio 0.5, fixed threshold — the setting where undefended damage is largest. *Retention*, $U = \overline{\text{gain}}_{\text{defended}} / \overline{\text{gain}}_{\text{undefended}}$, is the honest loop's own `honeypot_only` TPR gain over control on S0 (synthetic, ratios 0.05 and 0.2, fixed threshold), defended over undefended: how much of the honest loop's benefit a defense leaves intact. Every reported delta is paired by seed: for each of the 5 seeds, an arm's final-round metric minus the control's final-round metric at that same seed, then the mean, standard deviation and paired $t = \bar{d}/(s_d/\sqrt{5})$ over the 5 differences.

4.8 Statistical families

Table 2 names every family of hypothesis tests in this paper, the correction applied within it, and whether it was pre-specified (confirmatory) or swept after the fact (exploratory) — the distinction a Holm or Bonferroni correction is answering in each case, stated once rather than re-derived per section.

Permutation test, alongside every paired t-test. For every cell in the first three families above, we also run a paired sign-flip permutation test: exact enumeration of all 2^n sign patterns for $n \leq 12$ (every family here except the 15-seed follow-up), Monte Carlo with 10^4 resamples otherwise. This removes the normality assumption the t-test makes, at no additional data collection cost, computed from the same stored per-seed deltas (`experiments/phase0_stats_recomp`

Table 2: Every statistical family in this paper: what it tests, how many cells, how corrected, and whether pre-specified.

Family	Hypothesis	Cells	Seeds	Test	Correction	Status
36-cell grid	A1 damages RF/XGBoost TPR at this (ratio, jitter)	36	5	paired t + permutation	Holm, family of 36	Exploratory (full grid swept)
6-cell RF follow-up	RF's at-distance TPR damage generalises across jitter 0.3–1.5	6	15 (+10-new check)	paired t + permutation	Holm, family of 6	Confirmatory (pre-registered, Section 6)
Mechanism isolation	The label conflict, not volume or prior shift, drives A1's damage	4	15 (5 original + 10-new check, Section 6)	paired t + permutation	Bonferroni, family of 4	Confirmed by both tests (pre-specified arms, one fixed distance)
Defense sweeps	Setting X dominates a baseline on both recovery and retention	varies (D1's 3 × 5 grid, frontier comparisons)	5	seed-bootstrap dominance probability, not a t/permutation family	n/a (a resampling probability, not a corrected p-value)	Exploratory (grid search)

It surfaces a limitation worth stating plainly rather than only in a methods footnote: at $n = 5$, the two sign patterns that reproduce the observed effect exactly (all-positive and all-negative) already cover $2/2^5 = 0.0625$ of the permutation space, so *no* $n = 5$ cell in this paper can reach an exact two-sided permutation p below 0.0625 — a floor above even an uncorrected $\alpha = 0.05$ — regardless of effect size. This is a general property of $n = 5$ matched-pair sign tests (2^{1-n} is the minimum achievable exact p at n pairs), not a defect of any particular result, and it means the permutation test is not a strictly more conservative substitute for the t-test at this sample size: it is a less powerful one, reported for transparency. A paired t-test's and a permutation test's p-values track each other closely in rank regardless: Pearson $r = 0.875$ between $-\log_{10}(p_t)$ and $-\log_{10}(p_{\text{perm}})$ across the 71 cells in this paper with both values strictly positive. Families at $n = 15$ do not hit the $n = 5$ floor ($2^{-14} \approx 6 \times 10^{-5}$, well below the Monte Carlo resolution used) and their permutation p-values corroborate their t-test conclusions without qualification — this includes the mechanism-isolation family, which the $n = 5$ floor left genuinely unresolved by the permutation test until a targeted 15-seed re-run (the one exception made to this paper's otherwise-closed experiment set, pre-registered before running) confirmed it by both tests (Section 6). A permutation p-value of exactly 0 (no resampled sign pattern in 10^4 Monte Carlo draws matched or exceeded the observed effect) is reported as "$p < 0.0001$," never as "$p = 0$" — a zero count means the true p is below the resolution, not that it is exactly zero. Where the two tests disagree on a 5-seed cell, both are reported and neither is treated as silently superseding the other.

5 CICIDS2017 IS NEAR-DEGENERATE

Before any attack or defense result can mean anything, the dataset it is measured on has to be able to carry the claim. It cannot, on CICIDS2017, in the feature representation CICFlowMeter produces. Every family with usable volume, and benign traffic itself, has a near-identical twin on the other side of any partition boundary we tried.

Table 3: Median NN RMS distance, `trusted_eval` to `honeypot_pool`, by near-duplicate grid. The guard-(c) leakage threshold is 0.25. Row counts (n eval) are measured at grid 0.005 and fall further as the grid coarsens and more rows are removed.

Family	n eval @0.005	0.005	0.01	0.02	0.05
BENIGN	229,695	0.0065	0.0086	0.0108	0.0171
DoS Hulk	30,334	0.0055	0.0069	0.0113	0.0161
DDoS	17,383	0.0050	0.0063	0.0082	0.0131
DoS GoldenEye	1,579	0.0130	0.0132	0.0136	0.0177
FTP-Patator	1,010	0.0030	0.0036	0.0051	0.0142

Table 3 reports the median nearest-neighbour distance, in normalized feature space, from each held-out `trusted_eval` row to its nearest same-class `honeypot_pool` row, at four near-duplicate-removal grid resolutions. Every family with at least 500 `trusted_eval` rows — the bar below which a rate cannot be reported stably — sits at least 14× below the 0.25 leakage threshold at every grid, benign included (5th percentile 0.0007–0.0058 across the same range). Coarser deduplication moves the distances only by deleting rows; none approaches the threshold.

This is not an artifact of where the split falls. Pooling each family's rows across all three partitions, shuffling uniformly, and re-splitting at the same sizes reproduces the same distances to within a factor of 1.5 for benign and every high-volume family: BENIGN 0.0067 vs. 0.0078 shuffled, DDoS 0.0049 vs. 0.0038, DoS Hulk 0.0054 vs. 0.0040. No partition — burst-aware or otherwise (Appendix A) — can separate train from held-out here, because the distances are set by the feature distribution, not by the cut.

No family clears both the volume bar and a non-degenerate distance. Figure 2 shows every family with usable volume sits left of

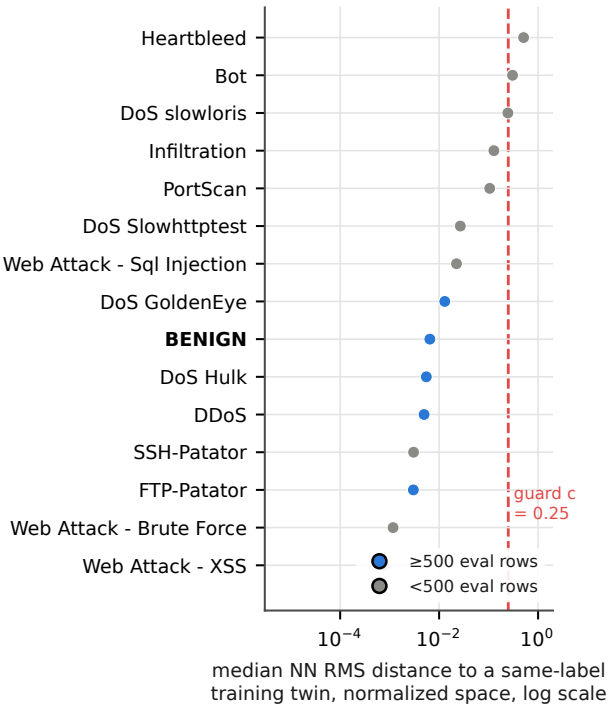


Figure 2: Median nearest-neighbour distance from `trusted_eval` to `honeypot_pool` by family (log scale, near-duplicate grid 0.005, the value used throughout this paper – Section 4). Grey: fewer than 500 `trusted_eval` rows, too few to report a stable rate. Every reportable family, benign included, sits well below the guard-(c) leakage threshold (dashed).

the threshold; the only families that sit to its right (Heartbleed, Bot, DoS slowloris near it) all have under 500 `trusted_eval` rows — Bot, the largest, has 193. There is no family that is simultaneously voluminous enough to report a stable rate on and distinguishable enough across a partition boundary to demonstrate learning rather than memorization.

We draw no S0 learning claim from CICIDS2017. A detector "improving" on this data would be rewarded for recognizing near-copies of specific held-out rows, not for learning a boundary that generalizes to traffic the loop never saw. CICIDS2017 instead answers two things this paper needs: the degeneracy measurement itself, and A1's damage-versus-distance curve in Section 6 — A1's poison is benign traffic stamped malicious, its damage is a function of a distance we set and directly measure, and this section's near-duplicate rate hands us the near end of that curve for free. S0's learning claim (Section 6) is made on the synthetic dataset instead, where entropy is controlled by construction and guard (c) passes cleanly.

6 THE ATTACK

6.1 S0: the honest loop works

Before any attack means anything, the clean loop has to improve the detector. On synthetic data, at poison ratio 0.2, it does, within one to two rounds: `honeypot_only` TPR — the arm the detector saw only through the decoy — rises from 0.39 (control) to 0.997 for RF and 0.34 to 0.994 for XGBoost. `seen_both` rises more modestly (0.946→0.961 RF, 0.950→0.963 XGBoost) and `seed_only` stays flat near its ceiling, untouched, as A4's controlled arm should be. This has a small cost even with no attacker present: fixed threshold FPR at 50% honeypot share rises to 0.024, the class-prior-shift cost of retraining on more positives.

6.2 A1: a cliff at copy-level fidelity

A1's damage on the FPR channel appears at exactly one place: copy-level mimicry fidelity, and only once enough of the training set is honeypot-derived. At raw benign fidelity (jitter 0, realized nearest-neighbour distance to trusted benign ≈ 0.011) fixed-threshold FPR rises catastrophically with poison ratio — RF +0.009/+0.025/+0.337/+0.880 and XGBoost +0.005/+0.014/+0.279/+0.563 at ratios 0.05/0.1/0.2/0.5 — but at every jitter setting we tested beyond that (realized distance ≥ 0.014), the same ratios sit inside 2σ of the control's own noise for both models, at every ratio tested (Figure 3). There is no gradient to read a distance threshold from: damage does not fall off gradually past the guard-(c) leakage line (0.25); it is already gone eighteen to twenty-five times closer to raw benign than that. *These numbers supersede an earlier, miscalibrated pass through this same sweep (Section 4); a threshold calibrated on a validation split full of near-copies of training rows read this cliff as a gentler, flatter effect than it is.* The median realized distance cannot actually resolve where this cliff sits: it moves by about 1% (0.0107 to 0.0108) across the exact jitter step, 0 to 0.002, where the damage collapses from catastrophic to inside the noise floor. *Twin fraction* — the share of a poison batch's rows whose nearest trusted-benign neighbour is within RMS distance 0.0015, a delta fixed from where the fifth-percentile of the realized-distance distribution jumps between those two jitters (Section 3) — does resolve it: 10.5% of the batch at jitter 0, 0.35% at jitter 0.002, exactly 0.0 at every jitter $\geq$ 0.005 tested. This tracks the FPR collapse point for point and is the x-axis Figure 3 now plots. None of these near-twins is a literal duplicate: checking the same pool against trusted_eval for exact float32 row matches, at every jitter including 0, finds zero. The jitter-0 damage is therefore driven by rows within 0.0015 RMS distance of a real benign row, not by byte-identical copies of one. Put plainly: on CICIDS2017, the false-positive channel fires only once the poison sits about as close to real benign traffic as CICIDS2017's own benign traffic already sits to itself — raw copy-level poison lands at realized distance $\approx$ 0.011, within a factor of $\sim$ 1.7 of benign's own median self-distance of 0.0065 (Section 5). The attacker does not need to exfiltrate a specific victim's traffic; replaying the pool's own benign feature statistics with no perturbation already puts one row in ten inside twin-fraction range, because CICIDS2017's benign class is itself narrow and repetitive. This channel is not a property of A1 alone; it is a property of how repetitive the target's benign traffic already is, and Section 8 returns to what that implies for where this attack does and does not generalize.

6.3 One mechanism, surfaced by the threshold policy

The same underlying mechanism — rows that sit in benign-populated feature space, stamped malicious by the auto-labeller — produces all of this section's damage. What differs is not the mechanism but which metric the threshold policy exposes it through: a fixed threshold reads the damage as false positives (Section 6, above); a recalibrated threshold reads the same underlying rows as lost detection, TPR. These were described as two separate channels in an earlier pass through this data; recalibrating does not remove the damage, it relocates which number shows it, so we no longer describe them as independent. Both surface at copy-level mimicry fidelity, once the statistics are held to the standard the rest of this paper uses. Naively reading off which of the 36 (model, ratio, jitter) cells clear an uncorrected $p < 0.05$ suggested TPR damage confined to RF and present at a distance as well as at copy fidelity; that reading does not survive correcting for the 36 simultaneous comparisons made across the grid. Applying a Holm-Bonferroni step-down correction (family-wise $\alpha = 0.05$) to all 36 cells, only three survive, and all three are at raw copy fidelity (jitter 0): RF at ratio 0.2 ($t = -20.3$) and ratio 0.5 ($t = -16.6$), and XGBoost at ratio 0.5 ($t = -8.5$) — XGBoost's own ratio-0.2, jitter-0 cell does not clear even its *uncorrected* threshold ($t = -2.6$, $p = 0.06$). A paired permutation test ranks these three cells the same way (Section 4), but at $n = 5$ every unanimous-sign cell, including all three survivors here, hits the same exact-test floor ($p_{\text{perm}} = 0.0625$) regardless of effect size, so the permutation test can confirm *which* cells are most extreme but not separate "clearly significant" from "borderline" among them the way the t-test's Holm correction does. **Every cell in this 36-cell family is read as exploratory, at $n = 5$, for this reason**: the confirmatory evidence in this paper for damage beyond copy-level fidelity is the 15-seed distance follow-up and the 15-seed mechanism isolation below, not this grid. Nothing at realized distance ≥ 0.25 survives within this 36-cell family, at either ratio, for either model (Table 5). **"Present at every distance tested," "significant near and significant far," and "the TPR effect is RF-only" — all stated in an earlier reading of this data — are withdrawn as claims made *from this family of tests*.** The corrected statement, at this correction level: TPR loss at copy-level fidelity is confirmed for RF at both poison ratios tested and for XGBoost at one of two. RF's raw means also show damage at large jitter (Figure 3, right column; Figure 4), and a separate, smaller, pre-registered follow-up resolves what the 36-cell family's power could not: RF, recalibrated TPR, jitter $\in \{0.3, 0.7, 1.5\}$ (realized median distance 0.25, 0.56, 1.05) at ratios $\{0.2, 0.5\}$, 15 seeds, Holm-corrected within this 6-cell family alone. **All six cells survive** (Table 4), exceeding the pre-registered confirmation bar (at least two adjacent jitters at both ratios). Because the original 5 seeds motivated this follow-up, testing them again inside the same confirmatory family would be mild double-dipping; we therefore also ran Holm correction on the 10 *new* seeds alone, and all six cells survive there too (p from 5.9×10^{-3} to 3.5×10^{-7}) — the result does not depend on the seeds that prompted it. A paired permutation test (Section 4) agrees: at 15 seeds this family does not hit the $n = 5$ exact-test floor, and four of six cells reach the Monte Carlo resolution limit ($p_{\text{perm}} < 0.0001$) while the remaining two ($p_{\text{perm}} = 0.0001$ and 0.0013) still clear the family's Holm thresholds — the t-test and permutation conclusions agree here without qualification. RF's TPR-loss damage is therefore confirmed across the far-jitter range tested, realized distance 0.25 to 1.05, not only at copy fidelity. This is specific to RF's recalibrated-threshold TPR: it does not extend to XGBoost (not tested in the follow-up family) or to the FPR channel under either model, which stays confirmed only at copy-level fidelity (Table 5). It also does not extend to jitter 0.01–0.10 (realized distance 0.014–0.10), which was tested only in the original 5-seed, 36-cell sweep and did not individually clear even an uncorrected threshold there — we do not have the seeds to say whether that range is damaging, null, or something else, and do not claim a dip or a monotonic trend across it. No mechanism is proposed for why the confirmed effect holds across 0.3–1.5 rather than only at copy fidelity; that is confirmed as real, not explained. XGBoost shows no comparable damaging effect anywhere on the original 36-cell grid; several of its mid-range points are individually significant before correction, but in the *improving* direction, and none of them survives correction either.

Table 4: RF, recalibrated TPR, at-distance follow-up: effect size and significance, 15 seeds and the 10 new seeds alone (Holm-corrected within each 6-cell family separately).

Jitter (dist.)	Ratio	ΔTPR, 15 seeds	ΔTPR, 10 new	t (15 / 10 new)
0.3 (0.25)	0.2	-0.100 ± 0.058	-0.114 ± 0.061	-6.68 / -5.87
0.3 (0.25)	0.5	-0.047 ± 0.051	-0.063 ± 0.055	-3.56 / -3.59
0.7 (0.56)	0.2	-0.126 ± 0.043	-0.134 ± 0.045	-11.28 / -9.45
0.7 (0.56)	0.5	-0.130 ± 0.074	-0.135 ± 0.073	-6.82 / -5.84
1.5 (1.05)	0.2	-0.116 ± 0.038	-0.123 ± 0.039	-11.75 / -10.06
1.5 (1.05)	0.5	-0.114 ± 0.032	-0.124 ± 0.030	-13.81 / -13.18

Table 5: One mechanism, two surfaces, after Holm correction. FPR/TPR-at-copy-fidelity: 36-cell grid family. RF-at-distance: separate, smaller, pre-registered 6-cell family (s26.6/s26.8).

Surface	Needs	Survives correction
FPR (fixed threshold)	Copy-level fidelity (twin fraction $\gg$ 0: 0.105 at raw jitter, 0.0 by jitter 0.005, Section 3); ≥10–20% poison	RF and XGBoost, both ratios tested
TPR loss (recalibrated threshold)	Copy-level fidelity, same regime as FPR	RF, both ratios; XGBoost, one of two ratios tested
TPR loss, RF, at distance (jitter 0.3–1.5, realized 0.25–1.05)	No copy-level fidelity needed; confirmed by a separate 15-seed, 6-cell family, and again on the 10 new seeds alone (Table 4)	RF, recalibrated threshold only, all 3 jitters, both ratios (not tested for XGBoost or for jitter 0.01–0.10)

The FPR channel's XGBoost trajectory is seed-dependent, and 5 seeds is too few to characterize it. At ratio 0.5, raw fidelity, fixed threshold, XGBoost's per-seed FPR at round 10 is bimodal, not clustered around the mean: three seeds climb (two reach 82–91% by round 10; the third climbs the same direction but later, to 69%), and two stay flat at 0.6–0.7%. RF climbs on every one of the 5 seeds. A mean-over-seeds table hides this — it reads as one noisy estimate of a common trajectory, and it is not. Every mean XGBoost number elsewhere in this paper should be read with that in mind; the full per-seed, per-round table for both models, including each seed's full round-by-round trajectory, is in Appendix A (Table 10).

6.4 Mechanism isolation

Three matched null controls (Section 4) isolate which part of A1 is doing the work. `a1truth` — A1's own rows, correctly labelled — removes the label conflict and the effect vanishes. `s0j` — attack rows carrying A1's jitter, stamped malicious — keeps the volume and the class-prior shift but not the conflict, and it too is null. `junk` — marginal-shuffled bulk, stamped malicious — tests volume alone and is also null. Only A1 itself, which keeps the label conflict, clears the control's own $\pm 2\sigma$ band (Figure 5). Prior shift and raw volume are not the mechanism; the label conflict is.

This result is confirmed at 15 seeds, by both tests. A1 clears the family's Bonferroni threshold ($\alpha = 0.0125$) by the paired t-test ($t = -11.3$, $p_t = 2.1 \times 10^{-8}$) and by the exact permutation test ($p_{\text{perm}} < 0.0001$), and every null arm's 95% CI includes 0 (`a1truth` $[-0.039, 0.014]$, `s0j` $[-0.040, 0.008]$, `junk` $[-0.021, 0.021]$). This is a pre-registered follow-up run after a 5-seed pilot left the result resting on the t-test alone, since the exact permutation test cannot clear even an uncorrected $\alpha = 0.05$ for any result in this paper at $n = 5$ (Section 4's floor); the qualitative finding is unchanged — the label conflict, not volume or prior shift, is the mechanism.

Section 6's broader 36-cell grid sweep, at 5 seeds, could not confirm a damaging effect at this same distance on its own — not because the effect is absent, but because that family's power was too low at that seed count, and every cell in it is read as exploratory for this reason, not as a confirmatory result in its own right. The confirmatory evidence for damage at this distance is the 15-seed distance follow-up (Section 6) and this 15-seed mechanism isolation, not the 36-cell sweep. The two readings of the single-distance result do not disagree with each other; the smaller, better-powered 15-seed families answer the question the 5-seed families lacked the power to settle either way.

7 DEFENSES

7.1 The policy-consistency blind spot

A generic defense that checks a honeypot label's consistency against the loop's own data fails here, and not because the label is correct. A1's poison is wrong against ground truth — benign rows stamped malicious — but it is *consistent with the defender's own labelling policy*, and once there is enough of it, that consistency is self-reinforcing: a model fit on the loop's data learns the stamped label as its own. Loss-based filtering (an auxiliary model's out-of-fold probability of the stamped label, dropping rows it scores low) demonstrates exactly this failure mode: it drops 60% of poison at 5% ratio, 33% at 10%, 1.5% at 20%, and 0% at 50% — it weakens precisely as the poison becomes dense enough to matter, and recovers essentially none of the damage anywhere damage exists (Table 6). It is not that the poison is correctly labelled; it is that a filter trained on the loop's own labels cannot see past a label the loop itself supplied consistently. kNN sanitization, anchored instead to the trusted `seed_train` labels rather than to the loop's own data, does not fail this way: it drops 98.5% of A1's poison, because it can see that the poison sits among *benign* seed neighbours despite carrying a malicious stamp. The blanket claim "generic defenses fail because the poison is policy-consistent" is therefore false — the failure is specific to defenses that learn from the data the loop itself produced, not to label-consistency checking in general.

Table 6: Loss-filter recovery of A1 damage by poison ratio (fixed threshold, CICIDS). No recovery is reported at 5%: there is no undefended damage there to recover.

Poison ratio	RF recovery	XGBoost recovery	Share of poison dropped
0.05	(no damage)	(no damage)	60%
0.10	−10%	18%	33%
0.20	2%	−1%	1.5%
0.50	0%	0%	0%

7.2 D1: cost-of-influence weighting

D1 prices a honeypot row's influence on the retrained model by what it cost the attacker to produce. Each row's *effort* is a geometric mean, over duration, packet count, byte count and protocol-state depth, of that row's cost relative to a median trusted benign flow's cost in the same component; its `sample_weight` saturates to 1 once effort clears a threshold E^* and is 0 for a free interaction. $E^* = 8, \gamma = 2$ — full influence costs roughly an order of magnitude more than a typical legitimate flow — was fixed, in DECISIONS.md, before D1 was implemented or any defended run existed. D1 was pre-registered to be judged against its own stated criterion: "not sensitive" if A1's FPR damage stays within 2σ of the control's noise floor across $E^* \geq 4, \gamma \geq 1$. **That criterion was not met.** Only 2 of the 12 configurations in that region satisfy it (both at $E^* = 32$); damage is monotone in both parameters, and while every configuration still removes at least 86% of the undefended damage, the residual at 50% poison scales with E^* and γ rather than vanishing. This is reported as a failed pre-registered test, and the criterion is not revised after seeing the result.

7.3 The frontier, and the right no-skill baseline

Recovery (R) and retention (U, Section 4) place every defense on one frontier (Figure 6). No setting of pre-registered D1 dominates kNN sanitization on this grid; the pre-registered point itself is the worst tested on synthetic retention. A defense using no cost information at all — a fixed weight on every honeypot row — is a real comparator here, not a strawman: at poison ratio 0.5 it matches kNN's recovery (98.9–99.8%) while keeping more of the honest loop's gain. It stops working once the poison ratio itself rises: at ratio 0.9 fixed uniform weighting recovers only 43–52%, where kNN and the best D1q setting both recover ~99%. A defense that

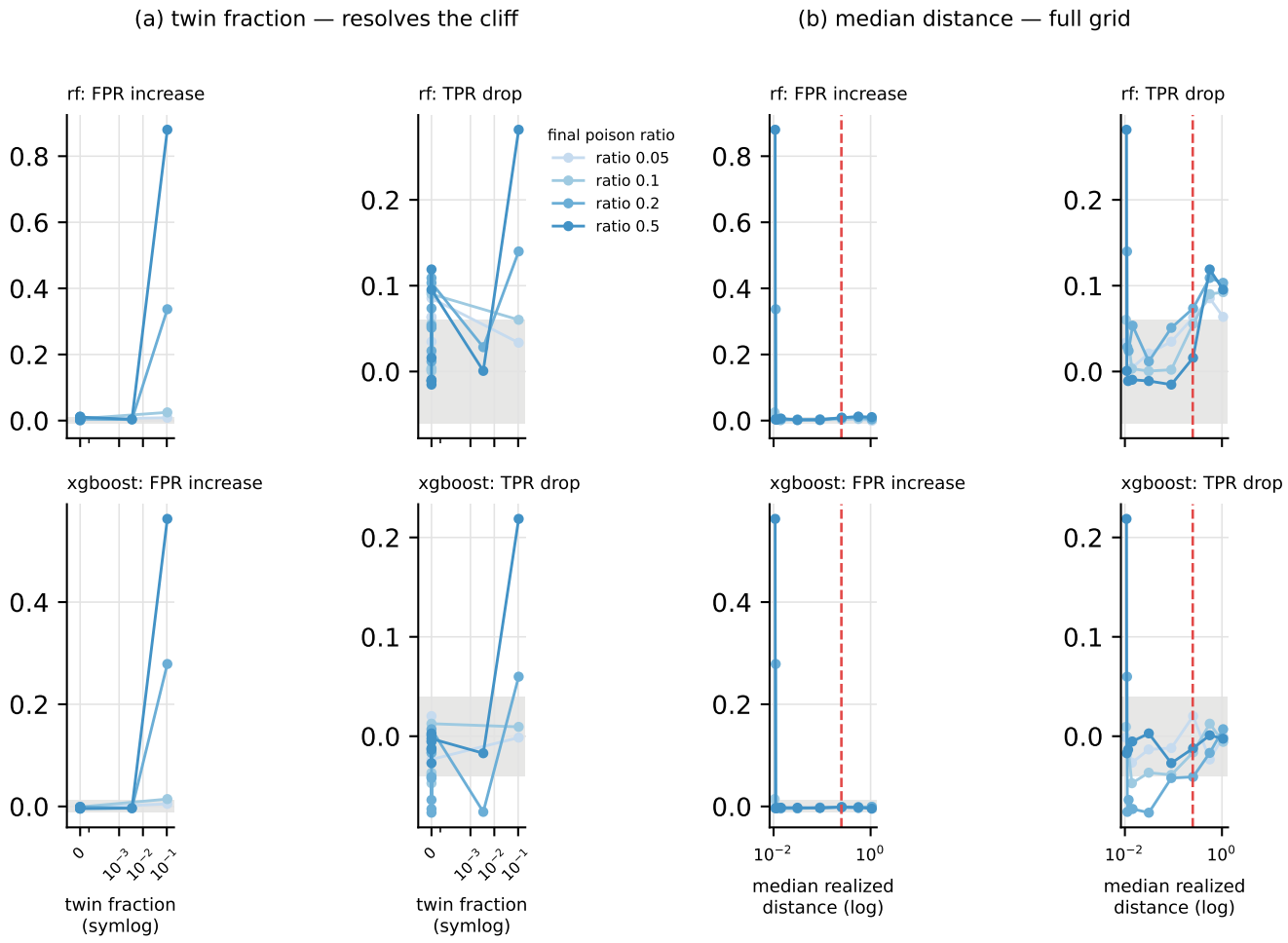


Figure 3: A1 damage against two views of mimicry fidelity, RF and XGBoost, FPR (fixed threshold) and TPR drop (recalibrated), at four poison ratios (light to dark; 0.2 and 0.5 carry two extra fine jitter steps). (a) Twin fraction — share of the poison batch within RMS 0.0015 of a trusted benign row (symlog, exactly zero for jitter ≥ 0.005). The FPR collapse to the noise floor tracks it point for point: 0.11 at raw fidelity, 0.003 one jitter step out, 0.0 beyond that. A coarser δ lags this collapse by one jitter step (Appendix A); the value plotted here does not. (b) Median realized distance (log), this paper's original capability axis: it still separates the far end of the grid (e.g. jitter 0.03 from 1.5), where twin fraction is uniformly zero, but it cannot resolve the cliff itself — it moves by about 1% across the same jitter step where the FPR damage collapses. RF's TPR drop is confirmed, after Holm correction, both at raw fidelity (36-cell grid family) and across jitter 0.3–1.5 (separate, better-powered 15-seed follow-up, Section 6); the FPR channel does not share this — it remains confirmed only at copy-level fidelity.

must work at a poison ratio the defender does not control cannot be a single fixed weight; it has to respond to how much of the training set the honeypot actually is. **ShareCap** — cap the honeypot's *effective* share of the training set at c, recomputed every retrain, rather than fix every row's weight — responds to exactly the quantity fixed uniform weighting cannot see. It is **undominated** by every D1 variant, kNN and fixed uniform weighting across all 87 (setting, retention-axis) comparisons tested, and it protects at both poison ratios tested, where fixed uniform weighting does not (Figure 6, bottom row). This is not a claim that ShareCap beats every other defense in every comparison — undominated means no alternative is at least as good on both recovery and retention, not that ShareCap is strictly better on each axis individually. The cap itself is a tuning knob: the full cap-sensitivity curve holds through roughly $c \leq 0.10$ for this model and dataset and degrades between $c = 0.10$ and $c = 0.20$; there is no cap that is simultaneously maximally protective and maximally retentive, and the safe range is model-and-dataset-dependent, not a universal constant. D1's measured overhead advantage stands regardless of the dominance result: 0.000216 s per round against kNN's 0.168 s at ratio 0.5, a 781× ratio (95% bootstrap 735–837×). ShareCap's overhead is of the same order as D1's — a weight recomputation, no auxiliary

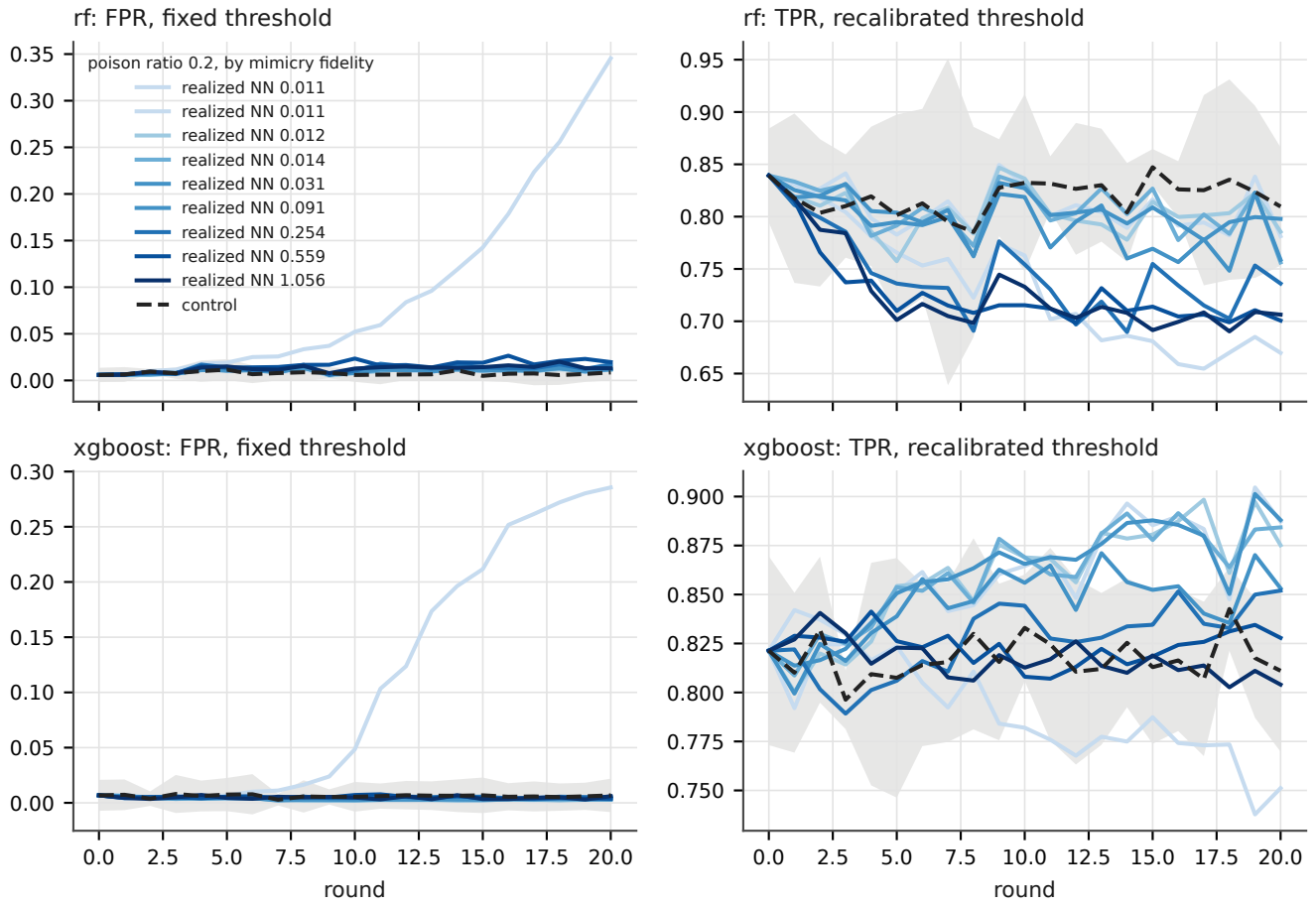


Figure 4: A1 on CICIDS2017 at poison ratio 0.2, per round: fixed-threshold FPR (left) and recalibrated-threshold TPR (right), RF and XGBoost, by realized mimicry fidelity (light to dark). Dashed: control mean ±2 sd across seeds. Only the two lowest-fidelity curves (realized NN 0.011, 0.014) separate from control on FPR — the FPR channel is confirmed only at copy-level fidelity. RF's TPR is different: confirmed both at the lowest-fidelity curve and, by a separate 15-seed follow-up (Section 6), across jitter 0.3–1.5 — visible here as every mid-to-high-fidelity curve moving together, which is now a confirmed effect, not raw-mean noise.

model, no per-row distance query — and needs no cost model, no trusted reference beyond the cap value itself, and no knowledge of the poison ratio. Against D1 specifically, an attacker forced to keep copy-level fidelity — where the FPR channel's damage lives, the surface this defense comparison measures (Section 6) — cannot pad individual flows to buy weight, because padding moves a flow away from the copy-level fidelity that damage depends on — a padded attack fails against the *undefended* loop too, so padding is not itself evidence for D1's protection. The available counter-move is volume: reaching the same damage under D1 costs 4–36× more flows, bracketed by the ratio grid tested (Appendix A).

8 DISCUSSION AND LIMITATIONS

When this matters. The attack needs benign traffic the adversary can reproduce with near-duplicate fidelity, and CICIDS2017 makes that cheap because its own benign class is narrow and repetitive (Section 5) — replaying the pool's own statistics with no perturbation already lands one row in ten within twin-fraction range (Section 6). Production networks with genuinely diverse, high-entropy benign traffic would raise the fidelity bar the attacker has to clear; networks that do not — IoT fleets, ICS traffic, automated service-to-service calls, anything whose "normal" is a small set of repeated request shapes — are closer to CICIDS2017's regime than to a diverse enterprise network, and are where this channel should be expected to be cheapest to exploit. We do not measure this on production traffic; it is a prediction from the mechanism, not a finding.

Practical advice. Cap the honeypot's effective share of the training set; it is the one defense here that costs nothing to deploy, needs no label information or trusted reference data beyond the cap value, and is undominated by every alternative we test (Section 7). Do not trust a label-consistency check that learns from the

Table 7: Defense comparison, XGBoost, 5 seeds, tau=0.1 calibration. Retention (synthetic) is reported at ratio 0.05 / 0.2; retention (CICIDS) at ratio 0.2. ShareCap's overhead is of the same order as D1's (a weight recomputation, no auxiliary model); not separately profiled.

Defense	Recovery r0.5	Recovery r0.9	Retention CICIDS 0.2	Retention synthetic 0.05 / 0.2	Overhead / round
Loss filter	−0.5%	n/a	97%	100% / 100%	0.68 s (56% of fit)
Uniform $w = 0.05$ (no skill)	99.8%	52%	71%	71% / 96%	negligible
Uniform $w = 0.1$ (no skill)	98.9%	43%	74%	91% / 98%	negligible
kNN sanitize	98.9%	99.1%	66%	42% / 78%	0.168 s
D1 (pre-registered)	92.9%	86.3%	66%	10% / 58%	0.000216 s
D1q (q=0.99, γ=2)	100%	98.7%	64%	67% / 94%	0.000216 s
D1fixed (no trusted data)	94.2%	n/a	65%	56% / 93%	0.000216 s
ShareCap ($c = 0.05$)	99.9%	100.0%	81%	100% / 99%	~D1

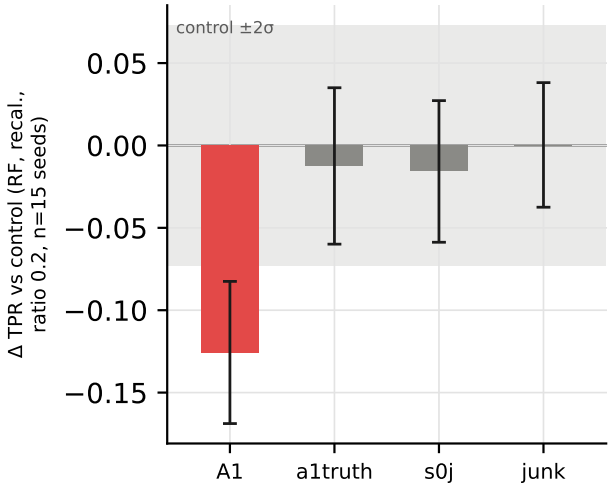


Figure 5: Mechanism isolation at one pre-specified distance (RF, recalibrated threshold, ratio 0.2, realized NN ≈ 0.56, jitter 0.7, $n = 15$ seeds): ΔTPR vs. control. A1: benign rows, jittered (not copy fidelity), stamped malicious. a1truth: the same rows, correctly labeled. s0j: genuine attack rows carrying A1's jitter, stamped malicious. junk: marginal-shuffled bulk, stamped malicious (run under the separate jitter-1.5 sweep). Error bars: sd over seeds. Band: the control's own ±2σ. Only A1 — the arm with the label conflict — moves outside it. Confirmed at 15 seeds by both tests: a1 clears the family's Bonferroni threshold ($\alpha/4 = 0.0125$) by the paired t-test ($t = -11.3$, $p = 2.1 \times 10^{-8}$) and by the exact permutation test ($p < 0.0001$), and every null arm's 95% CI includes 0 (a1truth $[-0.039, 0.014]$, s0j $[-0.040, 0.008]$, junk $[-0.021, 0.021]$). Pre-registered after a 5-seed pilot.

loop's own data to catch this poison — it cannot, because the poison is consistent with the loop's own labelling policy by construction; a check anchored to data outside the loop can.

Limitations. This is an offline simulation of the loop, not a deployed system: the adversary's batches are drawn from a fixed pool rather than generated against a live honeypot, and there is no network-level cost to reaching the decoy in the first place. We measure flow features only, not payload content, so any attack or defense that depends on packet contents is out of scope. **The attack is realized directly in flow-feature space, not demonstrated end to end from packets**: A1 perturbs feature rows (Section 4's $G(x, \sigma_j, \epsilon)$), it does not generate packets that a flow exporter then summarizes into those rows, so whatever imprecision a real packet-to-flow pipeline would add to an attacker's control over the realized distance is not measured here (Section 3's benign-distribution-knowledge axis states this assumption explicitly). CICIDS2017 answers the degeneracy measurement and the fidelity-versus-damage curve but carries no learning claim (Section 5); the synthetic dataset answers the honest-loop learning claim but has no attacker-cost profile of its own — **every quantitative result in this paper is centred on these two datasets and no other**. Five seeds is enough to establish some effects and not others: XGBoost's fixed-threshold FPR trajectory splits into two distinct per-seed groups rather than clustering around a mean (Section 6), and RF's damage at large mimicry distance needed a dedicated 15-seed follow-up to confirm — the original 5-seed, 36-cell grid family lacked the power to resolve it, even though the effect was real (Section 6). We only had the budget to run that follow-up for RF; the same question is open for XGBoost, and **the main grid throughout this paper is 5 seeds unless stated otherwise**. ShareCap's safe cap range is validated on neither a second dataset nor a third model (Appendix A has the full statement). Further limitations — the partition's file-order versus verified timestamp, and ShareCap's adaptive-attacker coverage — are also in Appendix A.

Future work. Four extensions this paper does not attempt but that the design points to directly: (1) *end-to-end packet realization* — generate the mimicry attack as actual packets through a real flow exporter, rather than perturbing feature rows directly, to measure how much control over realized distance survives that pipeline; (2) evaluating against Engelen et al. [7]'s reprocessed CICIDS2017, built specifically to correct the labeling and extraction errors Section 2 cites it for, to see whether the degeneracy measurement (Section 5) is a property of the original release or survives a cleaner re-extraction; (3) a second, independent NIDS dataset, to separate what in this paper's A1 results is specific to CICIDS2017 from what generalises; and (4) higher-powered grids for both models — the RF at-distance confirmation (Section 6) took 15 seeds to resolve what 5 could not, and the same question for XGBoost, and for the FPR channel at distance under either model, is open for exactly

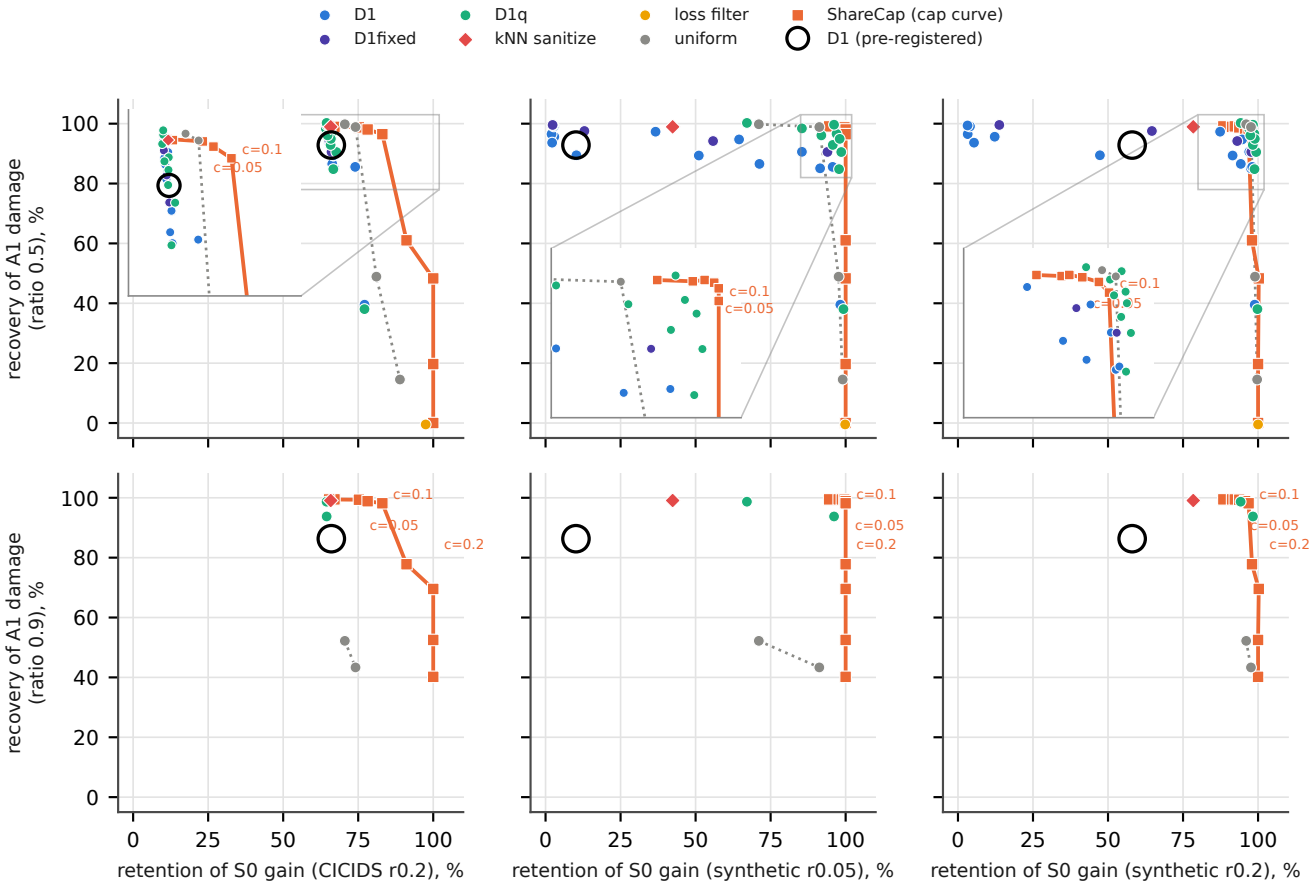


Figure 6: Recovery of A1 damage vs. retention of the honest loop's S0 gain, three retention axes (columns), at poison ratio 0.5 (top row) and 0.9 (bottom row; only configurations run at that ratio are plotted). Points: D1 variants, kNN sanitize, loss filter, fixed-weight uniform down-weighting (dotted: the no-skill line at each weight). Orange: ShareCap's full cap-sensitivity curve, $c \in \{0.01, \ldots, 0.5\}$, with c labeled at three points. Circled: the pre-registered D1 configuration ($E^* = 8, \gamma = 2$). Insets zoom the crowded high-retention corner of the top-row synthetic panels; the main axes keep their full 0–100% range, so the pre-registered D1 point stays visible outside the zoomed region. Uniform fails at ratio 0.9 (recovery 43–52%) where ShareCap does not; at ratio 0.5 alone they look similar.

the same reason: nothing ruled it out, we did not have the seeds to settle it.

9 CONCLUSION

Auto-labeling a honeypot's traffic and retraining a detector on it treats the label as free because the source is a decoy. It is not free whenever the attacker knows, or can reliably infer, the labelling policy: it can then choose exactly what the decoy sees, and the honeypot becomes a write channel into the defender's training set that the defender opened on purpose. We showed this channel is exploitable on a modern, flow-based retraining loop with a single mechanism — a label that contradicts its neighbourhood — that a fixed detection threshold reads as false positives and a recalibrated one reads as lost detection, not two separate effects. The false-positive surface needs near-copy mimicry fidelity; the detection-loss surface, for the one model where we confirm it generalises, does not, holding across a much wider fidelity range. Neither needs model access or knowledge of the poison ratio, and neither costs the attacker more than producing traffic that already resembles what the network generates — under the conditions we test, not as a formally zero cost. A generic label-consistency defense cannot see this poison because it agrees with the defender's own policy; a one-line cap on the honeypot's share of the training set can, and is Pareto-undominated over every recovery-retention configuration we evaluate it against — no alternative is at least as good on both axes — without needing any of the information those alternatives require. The honest version of this loop works — that was never in question — but the design that makes it convenient, a rule that trusts a decoy's traffic unconditionally once an attacker knows that is the rule, is the same design that makes it attackable, and the fix costs almost nothing to add.

Artifact availability. The code, every committed result CSV this paper's numbers trace to, and `DECISIONS.md` — the full methodology and findings log referenced throughout, kept as we worked rather than reconstructed afterward — are at https://github.com/harshitchhabi/IDS. Every figure in this paper regenerates from the committed CSVs with a single command (`python scripts/run.py figures`); no experiment needs to be re-run to reproduce a figure from its underlying numbers.

REFERENCES

[1] Ahmed Abdou, Ryan Sheatsley, Yohan Beugin, Tyler Shipp, and Patrick McDaniel. 2021. HoneyModels: Machine Learning Honeypots. In *MILCOM 2021 – 2021 IEEE Military Communications Conference (MILCOM)*.

[2] Marco Barreno, Blaine Nelson, Russell Sears, Anthony D. Joseph, and J. D. Tygar. 2006. Can Machine Learning Be Secure?. In *Proceedings of the 2006 ACM Symposium on Information, Computer and Communications Security (ASIACCS)*.

[3] Battista Biggio, Blaine Nelson, and Pavel Laskov. 2012. Poisoning Attacks against Support Vector Machines. In *Proceedings of the 29th International Conference on Machine Learning (ICML)*.

[4] Battista Biggio and Fabio Roli. 2018. Wild Patterns: Ten Years After the Rise of Adversarial Machine Learning. *Pattern Recognition* (2018).

[5] Robert A. Bridges, Thomas R. Mitchell, Mauricio Muñoz, and Ted Henriksson. 2025. SoK: Honeypots & LLMs, More Than the Sum of Their Parts? arXiv:2510.25939

[6] Antonio Emanuele Cinà, Kathrin Grosse, Ambra Demontis, Sebastiano Vascon, Werner Zellinger, Bernhard A. Moser, Alina Oprea, Battista Biggio, Marcello Pelillo, and Fabio Roli. 2023. Wild Patterns Reloaded: A Survey of Machine Learning Security against Training Data Poisoning. *Comput. Surveys* (2023).

[7] Gints Engelen, Vera Rimmer, and Wouter Joosen. 2021. Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study. In *2021 IEEE Security and Privacy Workshops (SPW)*. 7–12.

[8] Wenjun Fan, Zichen Yang, Yuanzhen Liu, Lang Qin, and Jia Liu. 2024. HoneyLLM: A Large Language Model-Powered Medium-Interaction Honeypot. In *Information and Communications Security (ICICS 2024) (LNCS, Vol. 15057)*.

[9] Linan Huang and Quanyan Zhu. 2019. Adaptive Honeypot Engagement through Reinforcement Learning of Semi-Markov Decision Processes. In *Decision and Game Theory for Security (GameSec 2019)*. https://doi.org/10.1007/978-3-030-32430-8_13

[10] Marius Kloft and Pavel Laskov. 2012. Security Analysis of Online Centroid Anomaly Detection. *Journal of Machine Learning Research* 13 (2012), 3681–3724.

[11] Phani Lanka, Khushi Gupta, and Cihan Varol. 2024. Intelligent Threat Detection—AI-Driven Analysis of Honeypot Data to Counter Cyber Threats. *Electronics* 13, 13 (2024), 2465.

[12] Maxime Lanvin, Pierre-François Gimenez, Yufei Han, Frédéric Majorczyk, Ludovic Mé, and Eric Totel. 2022. Errors in the CICIDS2017 Dataset and the Significant Differences in Detection Performances It Makes. In *Risks and Security of Internet and Systems (CRiSIS 2022)*.

[13] Mohammed Shaad Mehboob Matcheswala and Amir Javed. 2024. Creating an Adaptive Defense Architecture Using an Adaptive Honeypot Algorithm and Network Traffic Classifier. In *AI Applications in Cyber Security and Communication Networks (ICCS 2023) (Lecture Notes in Networks and Systems, Vol. 1032)*.

[14] Blaine Nelson, Marco Barreno, Fuching Jack Chi, Anthony D. Joseph, Benjamin I. P. Rubinstein, Udam Saini, Charles Sutton, J. D. Tygar, and Kai Xia. 2008. Exploiting Machine Learning to Subvert Your Spam Filter. In *Proceedings of the 1st USENIX Workshop on Large-Scale Exploits and Emergent Threats (LEET)*.

[15] James Newsome, Brad Karp, and Dawn Song. 2006. Paragraph: Thwarting Signature Learning by Training Maliciously. In *Recent Advances in Intrusion Detection (RAID 2006) (LNCS, Vol. 4219)*. 81–105.

[16] Hakan T. Otal and M. Abdullah Canbaz. 2024. LLM Honeypot: Leveraging Large Language Models as Advanced Interactive Honeypot Systems. arXiv:2409.08234

[17] Andrea Paudice, Luis Muñoz-González, and Emil C. Lupu. 2018. Label Sanitization against Label Flipping Poisoning Attacks. arXiv:1803.00992 ECML-PKDD Nemesis workshop, 2018.

[18] Roberto Perdisci, David Dagon, Wenke Lee, Prahlad Fogla, and Monirul Sharif. 2006. Misleading Worm Signature Generators Using Deliberate Noise Injection. In *2006 IEEE Symposium on Security and Privacy*.

[19] Arnaud Rosay, Eloïse Cheval, Florent Carlier, and Pascal Leroux. 2022. Network Intrusion Detection: A Comprehensive Analysis of CIC-IDS2017. In *Proceedings of the 8th International Conference on Information Systems Security and Privacy (ICISSP)*. 25–36.

[20] Benjamin I. P. Rubinstein, Blaine Nelson, Ling Huang, Anthony D. Joseph, Shing-hon Lau, Satish Rao, Nina Taft, and J. D. Tygar. 2009. ANTIDOTE: Understanding and Defending against Poisoning of Anomaly Detectors. In *Proceedings of the 9th ACM SIGCOMM Conference on Internet Measurement (IMC)*.

[21] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. 2018. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP 2018)*, Vol. 1. SciTePress, 108–116. https://doi.org/10.5220/0006639801080116

[22] Shreyas Srinivasa, Jens Myrup Pedersen, and Emmanouil Vasilomanolakis. 2023. Gotta Catch 'em All: A Multistage Framework for Honeypot Fingerprinting. *ACM Digital Threats: Research and Practice* 4, 3 (2023), 42.

[23] Sebastián Vargas Yáñez and Sergio Tobón. 2026. Hive-AI: A Defended Multi-Service Honeypot Framework for Generative AI APIs. *International Journal of Information Security* 25 (2026), 158. https://doi.org/10.1007/s10207-026-01307-0

[24] Ziyang Wang, Jianzhou You, Haining Wang, Tianwei Yuan, Shichao Lv, Yang Wang, and Limin Sun. 2024. HoneyGPT: Breaking the Trilemma in Terminal Honeypots with Large Language Model. arXiv:2406.01882

[25] P. E. Yugai and D. A. Moskvin. 2024. Using Machine Learning Algorithms and Honeypot Systems to Detect Adversarial Attacks on Intrusion Detection Systems. *Automatic Control and Computer Sciences* 58 (2024), 1226–1233.

A APPENDIX

A.1 Twin fraction: sensitivity to the chosen delta

Table 8: Twin fraction by jitter, at three values of δ.

Jitter	$\delta = 0.001$	$\delta = 0.0015$	$\delta = 0.003$
0.000	0.057	0.105	0.212
0.002	0.00002	0.003	0.173
0.005	0.000	0.000	0.001
$\geq$0.01	0.000	0.000	0.000

Figure 3(a) uses $\delta = 0.0015$, chosen from where the realized-distance distribution's 5th percentile jumps between jitter 0 and jitter 0.002 (Section 3). Table 8 recomputes twin fraction at $\delta \in \{0.001, 0.003\}$, bracketing that choice by roughly 2x in each direction. The qualitative story — a cliff within one to two jitter steps of raw fidelity, not a gradient — holds at all three, but the exact location is not delta-invariant: $\delta = 0.001$ and $\delta = 0.0015$ both collapse to near-zero by jitter 0.002, matching where the FPR damage itself disappears (s26.2); $\delta = 0.003$ is coarse enough that it still reads 0.173 at jitter 0.002 — comparable to $\delta = 0.0015$'s own raw-jitter value — and does not fall to near-zero until jitter 0.005, one step later than the damage does. A delta chosen too loosely lags the effect it is meant to track; the finer two values, including the one used in Figure 3, do not.

A.2 Guard (a): near-duplicate grid sweep, and why `leak_warning` is attack-only

The near-duplicate guard (Section 4) removes a large fraction of the benign class on CICIDS2017 at any reasonable grid resolution, so that resolution cannot be picked by feel. Sweeping `near_dup_grid` $\in$ {off, 0.005, 0.01, 0.02, 0.05}: with the guard off, partition construction fails outright (CICIDS2017 has roughly 12.3k flows that are row-content-identical, to six decimal places, across `seed_train` and `trusted_eval`). From 0.005 to 0.05, benign rows removed range over 2× and `seed_train` shrinks from 456k to 267k rows, but RF AUROC stays in 0.98–0.996 and TPR in 0.95–0.97 throughout; `leak_warning`

is set at every grid tested. The result does not depend on which grid in this range is used, and the guard is necessary regardless of grid: this paper uses 0.005 throughout.

`leak_warning` itself is gated on the attack class alone, not benign. Guard (c)'s purpose in setting that flag is to catch `S0` passing its checkpoint by memorization of a specific held-out row rather than by learning a boundary; two different benign flows, unrelated to any attack, can legitimately carry identical flow statistics (the same web request shape occurring twice), so a near-zero benign nearest-neighbour distance is not by itself evidence of a leak the way the same distance on the attack class would be. This does not mean the benign side is ignored: guard (c) measures it on the same scale and reports it unconditionally, and Section 5 is built entirely on that benign-side measurement, because it bears directly on A1 (whose poison and whose measured damage are both benign-side) and on the threshold-calibration fix (Section 4).

A.3 Burst splitting

Section 4's guard (b) is a row-level cut, which could in principle slice through the middle of a contiguous burst of highly similar flows (a flood tool's output, most obviously). Burst-level splitting tests this directly: segment each (day, family) stream into contiguous episodes by inter-flow gap (> 2s, chosen from the measured gap distribution) and assign each burst whole to one partition, never dividing one across a boundary (`tests/test_partition.py` asserts this invariant on the reconstructed union of all four partitions). Two family groups emerge: session-like families (Bot, SSH-/FTP-Patator, DoS GoldenEye, Web Attack variants) have a median inter-flow gap of 4–38s and thousands of bursts per day, where a row-level cut genuinely could split one; continuous floods (DoS Hulk, DDoS, DoS Slowhttptest, PortScan) have a median gap under 1s and essentially do not pause — DDoS is ~128k rows in 22 bursts for the entire day.

With burst splitting on, the guard-(a) grid sweep repeats at the same order of magnitude as the row-level cut (nn_attack_p5 0.0010–0.0058 across grid 0.005–0.05, versus 0.0010–0.0067 row-level, Section 5); `leak_warning` is set at every grid, unchanged. The per-family nearest-neighbour breakdown explains why burst-awareness does not help: DoS Hulk's nearest `honeypot_pool` neighbour to a `trusted_eval` row has RMS distance 0.0003, DDoS 0.00017, DoS GoldenEye 0.00017 — a different burst, sometimes a different half of the day, and the feature vector is still within noise of zero distance. The degeneracy (Section 5) is a property of the feature representation these families produce, not an artifact of where or how the cut is drawn.

A.4 D1: full definition and sensitivity grid

D1 prices a honeypot row's influence by attacker effort. Per-flow cost is $x_i = (d_i, p_i, b_i, s_i)$ — duration, packet count, byte count, and protocol-state depth $s_i = \min(\text{fwd packets}, \text{bwd packets})$. Each component is scaled by its reference r_k, the median of that component over `seed_train`'s benign rows (trusted, defender-owned): effort is the geometric mean $e_i = \left[\prod_k (1 + x_{ik}/r_k)^{1/4}\right] - 1$, so $e_i = 0$ only if every component is zero, $e_i = 1$ for a flow that costs a median benign flow's worth in every component, and e_i is invariant to the units of each component. Weight saturates as $w_i = \min(1, (e_i/E^*)^\gamma)$; the pre-registered default $E^* = 8, \gamma = 2$ gives a median benign flow weight $(1/8)^2 = 0.016$ — full influence costs roughly an order of magnitude more effort than a typical legitimate flow.

Table 9: D1 FPR-rise sensitivity grid, XGBoost, ratio 0.5, raw fidelity. Bold: inside $2\sigma_{\text{control}}$.

$\gamma \backslash E^*$	2	4	8	16	32
1	+0.340	+0.081	+0.053	+0.030	+0.015
2	+0.084	+0.060	+0.040	+0.025	**+0.005**
3	+0.076	+0.059	+0.036	+0.020	**+0.003**

Table 9 is the full pre-registered sensitivity grid (Section 7 reports only the pass/fail count and the extremes). XGBoost, ratio 0.5, jitter 0 (raw fidelity, undefended damage +0.563). Bold cells are inside $2\sigma_{\text{control}}$ (0.012), the pre-registered "not sensitive" band.

Only 2 of the 12 configurations in the pre-registered region ($E^* \geq 4, \gamma \geq 1$) clear the band, both at $E^* = 32$; damage is monotone in both parameters, and even the weakest configuration in the region ($E^* = 4, \gamma = 1$) still removes 86% of the undefended damage — D1 is insensitive in *direction* and sensitive in *magnitude*. A component ablation at the default ($E^* = 8, \gamma = 2$) shows the equal-weight geometric mean is not the best available choice on this data: packets-only leaves a residual of +0.011 (inside the band), depth-only +0.015, bytes-only +0.049, duration-only +0.066, all four combined +0.040 — heavy-tailed benign flows with large byte counts or duration reach full weight under the combined effort more easily than packet count or protocol depth would let them. This is reported as a property of the data, not acted on: the pre-registered default is not re-tuned after seeing it.

A.5 XGBoost per-seed trajectories

Section 6 reports XGBoost's fixed-threshold FPR response to A1 as seed-dependent rather than characterisable from a mean over 5 seeds; Table 10 is the full per-seed trajectory this is based on (ratio 0.5, raw fidelity, rounds 2 through 20). XGBoost is bimodal: three seeds climb (2 and 4 early, reaching 78–91% FPR over rounds 8–10; seed 3 the same direction but later, 5.3% at round 6, 53.8% at round 8, 69.3% at round 10); two seeds stay flat (1 and 5, both under 1% through round 10, though seed 5 edges up to 11.7% by round 20 while seed 1 stays under 2%). RF climbs on all 5 seeds, with none staying flat. A mean-over-seeds summary hides this: XGBoost's round-10 mean is dragged up by three seeds while two show essentially no effect, which is a bimodal outcome, not a noisy estimate of one common trajectory. Every mean XGBoost number elsewhere in this paper should be read with that in mind.

A.6 Cost padding: the adversary's counter-move that does not work

Section 7 states that an attacker forced to keep copy-level fidelity cannot pad individual flows to buy D1 weight, because padding moves a flow away from that fidelity. Table 11 is the measurement this rests on: `cost_padding`= m scales duration, packets and bytes before jitter is applied, XGBoost, ratio 0.5.

Table 10: Fixed-threshold FPR by seed and round, ratio 0.5, raw fidelity, both models.

Model	Seed	r2	r4	r6	r8	r10	r12	r15	r20
XGBoost	1	0.005	0.005	0.004	0.006	0.006	0.006	0.007	0.019
XGBoost	2	0.016	0.503	0.737	0.782	0.823	0.878	0.924	0.955
XGBoost	3	0.006	0.007	0.053	0.538	0.693	0.714	0.734	0.800
XGBoost	4	0.046	0.707	0.776	0.839	0.908	0.949	0.948	0.958
XGBoost	5	0.005	0.004	0.004	0.006	0.007	0.008	0.010	0.117
RF	1	0.048	0.232	0.618	0.721	0.767	0.804	0.858	0.911
RF	2	0.032	0.153	0.503	0.700	0.735	0.776	0.840	0.894
RF	3	0.021	0.088	0.362	0.612	0.706	0.743	0.776	0.853
RF	4	0.015	0.095	0.383	0.615	0.728	0.740	0.788	0.869
RF	5	0.042	0.284	0.647	0.729	0.779	0.836	0.880	0.917

Table 11: Padding vs. D1: realized fidelity, weight, and FPR rise, XGBoost, ratio 0.5.

m	Realized NN	Undefended: weight / ΔFPR	D1: weight / ΔFPR	Attacker bytes
1	0.011	1.00 / +0.563 ± 0.457	0.23 / +0.040 ± 0.043	3.2×10^8
2	0.135	1.00 / +0.021 ± 0.020	0.31 / +0.007 ± 0.004	6.4×10^8
8	0.331	1.00 / −0.003	0.48 / −0.001	2.6×10^9
32	0.532	1.00 / −0.005	0.90 / −0.004	1.0×10^{10}

Padding does raise D1's mean weight (0.23 to 0.90 from $m = 1$ to $m = 32$), but it moves the poison's realized distance from 0.011 to 0.53 — well past where A1's damage lives at all (Section 6) — and the damage is gone regardless of defense. Critically, the padded attack *also fails against the undefended loop* (ΔFPR already down to +0.021 at $m = 2$): padding is not evidence for D1's protection, because the fidelity break does that on its own, with or without a defense installed. The attacker's effective counter-move to D1 is volume, not padding (Section 7: 4–36× more copy-level flows). Padding only demonstrates that the cheaper evasion is not available.

A.7 Additional limitations

Beyond the four limitations in Section 8:

- **The partition is by file order, not a verified capture timestamp.** CICIDS2017's public `MachineLearningCVE` release carries no timestamp column (Section 4), so "day split" in this paper means the dataset's own file boundaries, not confirmed calendar time. A burst-aware split gives comparable results (Appendix A, "Burst splitting"), but that is evidence the result survives a different file-order cut, not independent evidence of separation on real wall-clock time.
- **ShareCap's safe cap range is not a universal constant.** It is measured for one model and one dataset at a time (Section 7); a different model or dataset may have a different safe range, and we do not have a way to predict it without measuring it.
- **No adaptive attacker against ShareCap beyond filling the cap.** We do not test an attacker who observes or infers the cap and tunes its poison ratio specifically around it, e.g. staying just under the cap indefinitely rather than pushing past it. Session-level cost-of-influence weighting (a defense that might address a session-aware version of this) was considered and parked, not run: the dataset that would carry the source IPs and timestamps it needs is fragmented and was unavailable to us.